Übungsblatt 5: Serielle Bussysteme

Mikrocomputertechnik 3 · Bitfolgen, Timing und Sensortransaktionen auf Papier

Musterlösung

Ziel und Arbeitsweise

Alle sechs Pflichtaufgaben sind ohne Platine oder Simulator auf Papier lösbar. Sie rekonstruieren Rahmen und Transaktionen, berechnen Zeit- und Stromgrenzen und unterscheiden gültige Daten von einem fehlgeschlagenen Transfer.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht sind 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgaben 4–5 (15 min), Aufgabe 6 (15 min). Lösungsbesprechungen gehören zu den Vorlesungsphasen.

UART bedeutet Universal Asynchronous Receiver-Transmitter, SPI Serial Peripheral Interface und I²C Inter-Integrated Circuit. Für direkte Pegeldiagramme gelten ideal LOW 0 V und HIGH 3,3 V. Geben Sie Rechenweg, Einheiten und Modellgrenzen an. Hexadezimale Werte beginnen mit `0x`. Alle Zahlenbeispiele sind Lehrannahmen.

Aufgabe 1: UART-Byte, Rahmen und Nutzrate (6 Punkte)

Ein UART arbeitet mit 115200 Baud, einem Bit je Symbol und dem Format 8N1: acht Datenbits, keine Parität, ein Stopbit. Idle ist HIGH, Start LOW. Daten laufen mit LSB, Least Significant Bit oder niedrigstem Bit, zuerst.

1. Nach dem Start folgen $1, 1, 0, 0, 1, 0, 1, 0$. Bestimmen Sie die übliche schriftliche Binärdarstellung mit höchstem Bit links und den Hexwert. Zeichnen Sie Start, Daten und Stop. (2 P)
2. Berechnen Sie Bitdauer, Rahmendauer und maximale Nutzdatenrate in Byte pro Sekunde bei lückenlosen Rahmen. Warum sind 115200 Byte pro Sekunde keine passende Antwort? (3 P)
3. Welche Parität entsteht bei Even Parity für dieses Byte? Wie viele Intervalle besitzt der entsprechende 8E1-Rahmen? (1 P)

Musterlösung 1

1. Datenstellen d_0 bis d_7 sind $1, 1, 0, 0, 1, 0, 1, 0$. In schriftlicher Reihenfolge d_7 bis d_0 ergibt das `0101 0011`, `0x53`, dezimal 83. Der Rahmen ist LOW-Start, diese acht Datenintervalle, HIGH-Stop. Die zeitliche Folge einfach als MSB-Schreibweise zu lesen ergäbe fälschlich `0xCA`. (1 P Zahl, 1 P korrektes Diagramm)

2. $T_b = 1/115200 \approx 8{,}681\,\mu\text{s}$. $T_f = 10T_b \approx 86{,}806\,\mu\text{s}$. $r = 115200/10 = 11520$ Byte/s. Ein Baud trägt hier ein Bit, nicht ein Byte; außerdem brauchen Start und Stop Zeit. Acht Datenbits ergeben 92160 Nutzdatenbit/s. (je 1 P)
3. Vier Einsen sind bereits gerade: Paritätsbit 0. Mit Start, acht Datenbits, Parität und Stop sind es elf Intervalle. (1 P)

Diese Obergrenze enthält weder Anwendungskopf noch zusätzliche Sendepausen.

Aufgabe 2: Samples und endlicher Empfangspuffer (5 Punkte)

1. Ein idealer 8N1-Empfänger erkennt die Startflanke genau. Sein Stop-Sample liegt bei $9{,}5T_r$, das Sender-Stopintervall zwischen $9T_s$ und $10T_s$. Prüfen Sie $T_r = 1{,}03T_s$ und $T_r = 1{,}06T_s$. Kalibriert der nächste Start automatisch die Oszillatorfrequenz? (2 P)
2. Ein RX-FIFO mit sechzehn Byteplätzen enthält unmittelbar nach einem Rahmenabschluss bei $t = 0$ acht Bytes. Weitere 8N1-Rahmen folgen bei 115200 Baud lückenlos; es wird nichts entnommen. Bestimmen Sie Belegung und Zeitpunkt nach acht sowie nach neun weiteren Abschlüssen. Formulieren Sie eine konservative Servicefrist. (3 P)

RX heißt Receive, Empfang, FIFO First In, First Out. T_s und T_r sind Sender- beziehungsweise Empfänger-Bitdauer.

Musterlösung 2

1. $9{,}5 \cdot 1{,}03T_s = 9{,}785T_s$ liegt im idealen Stopintervall. $9{,}5 \cdot 1{,}06T_s = 10{,}07T_s$ liegt außerhalb; bei direkt folgendem Rahmen kann dort dessen LOW-Start stehen. Die nächste Startflanke richtet die Phase neu aus, kalibriert aber nicht automatisch den Oszillator. Das ist keine universelle UART-Toleranzgrenze. (1 P Samples, 1 P Phase/Takt-Abgrenzung)
2. Acht weitere Bytes füllen die freien Plätze: $8T_f = 80/115200 \approx 694{,}4\,\mu\text{s}$, Belegung 16. Der neunte weitere Abschluss bei $9T_f = 90/115200 = 781{,}25\,\mu\text{s}$ verursacht ohne Entnahme Overrun. „Voll“ ist also nicht bereits „überlaufen“. Konservativ vor $8T_f$ Platz schaffen, einschließlich begrenzter zusätzlicher Bearbeitungsverzögerung. Die bekannte Abschlussphase ist eine explizite Annahme dieser Rechnung. (je 1 P)

Aufgabe 3: SPI-Austausch, Timing und Nutzdaten (6 Punkte)

1. Zwei Acht-Bit-Modellregister enthalten anfangs Controller `0x96`, Target `0x3C`. Sie schieben MSB zuerst nach links und nehmen pro Takt das zuvor erfasste obere Gegenbit rechts auf. Bestimmen Sie beide Register nach einem, vier und acht Takten. (2 P)
2. Modus 0: Welche Flanke tastet ab, welche bereitet das nächste Bit vor? Bereitstellung 18 ns, zusätzlicher Versatz 7 ns, Setup 10 ns. Reicht eine halbe Taktperiode bei 10 MHz, bei 20 MHz? (2 P)
3. Ein Spielprotokoll braucht ein Kommandobyte und drei Antwortbytes. Der Controller sendet für die Antwort drei Dummy-Bytes. Berechnen Sie Taktzahl, Transferzeit bei 10 MHz und Nutzdatenrate. Zählen Dummy- und Antwortbytes als getrennte aufeinanderfolgende Bytezeiten? (1 P)
4. Zehn Targets teilen SCK, MOSI und MISO und besitzen je ein CS. Wie viele Controller-Signalpins sind nötig? Welche Bedingung muss für die nicht ausgewählten MISO-Ausgänge gelten? (1 P)

MSB bedeutet Most Significant Bit, höchstes Bit. SCK ist Serial Clock, MOSI Master Out Slave In, MISO Master In Slave Out, CS Chip Select. GND und Versorgung zählen zusätzlich.

Musterlösung 3

1. Nach einem Takt: `0x2C` und `0x79`. Nach vier: `0x63` und `0xC9`. Nach acht: `0x3C` und `0x96`. Beide Eingangsbits werden vor beiden Registerupdates erfasst. Das sind Rechenzustände, keine garantiert auslesbaren Register einer Hardware. (1 P korrekte Schieberegel, 1 P Zwischen-/Endwerte)
2. Modus 0: steigend abtasten, fallend nächste Daten vorbereiten. $18 + 7 + 10 = 35$ ns. Bei 10 MHz ist $T/2 = 50$ ns, Reserve 15 ns. Bei 20 MHz sind es nur 25 ns: Budget verletzt. Hold-Zeit und Auswahlgrenzen sind damit noch nicht vollständig geprüft. (je 1 P)
3. Vier Bytezeiten sind 32 Takte, 3,2 μs. Drei Nutzdatenbytes ergeben 937500 Byte/s. Dummy-Daten hinaus und Antwort herein laufen gleichzeitig; hier sind es nicht sieben Bytezeiten. (1 P)
4. $3 + 10 = 13$ Signalpins. Nicht ausgewählte Targets müssen MISO entsprechend ihrem Gerätevertrag freigeben; zwei gegensätzliche aktive Rückpfadtreiber wären ein Konflikt. (1 P)

Aufgabe 4: I²C-Pull-ups dimensionieren (5 Punkte)

Gegeben: Versorgung $V_{\mathrm{IO}} = 3{,}3$ V, obere LOW-Spannung $V_{\mathrm{OL,max}} = 0{,}4$ V, zulässiger Sinkstrom $I_{\mathrm{OL}} = 3$ mA, Buskapazität $C_B = 100$ pF und maximale 30–70-Prozent-Anstiegszeit $t_{r,\mathrm{max}} = 300$ ns. Verwenden Sie

$$R_{\mathrm{min}} = \frac{V_{\mathrm{IO}} - V_{\mathrm{OL,max}}}{I_{\mathrm{OL}}}, \qquad t_r = 0{,}8473 R_P C_B.$$

1. Bestimmen Sie untere und obere Widerstandsgrenze. (2 P)
2. Prüfen Sie 2,2 kΩ und 4,7 kΩ anhand der Anstiegszeit. (1 P)
3. Zwei Platinen bringen jeweils 2,2 kΩ parallel an dieselbe Leitung. Welcher Gesamtwiderstand und LOW-Strom entstehen bei $V_{\mathrm{OL}} = 0{,}4$ V? Passt dies noch zu den Vorgaben? (1 P)
4. Nun beträgt $C_B = 400$ pF bei sonst unveränderten Grenzen. Existiert überhaupt noch ein zulässiger Widerstandsbereich? Welche Aussage widerlegen die Zahlen? (1 P)

Musterlösung 4

1. $R_{\mathrm{min}} = 2{,}9/0{,}003 \approx 966{,}7\,\Omega$. $R_{\mathrm{max}} = 300\,\mathrm{ns}/(0{,}8473 \cdot 100\,\mathrm{pF}) \approx 3540{,}7\,\Omega$. (je 1 P)
2. 2,2 kΩ: $t_r \approx 186{,}4$ ns, passend. 4,7 kΩ: $t_r \approx 398{,}2$ ns, zu langsam. Beide liegen über der unteren Stromgrenze. (1 P)
3. $R_P = 1{,}1$ kΩ, $I_{\mathrm{OL}} = 2{,}9/1100 \approx 2{,}636$ mA; $t_r \approx 93{,}2$ ns. Der Gesamtwiderstand liegt hier noch zwischen beiden Grenzen. Weitere parallele Widerstände müssten neu geprüft werden. (1 P)
4. Die obere Grenze sinkt auf ungefähr $885{,}2\,\Omega$, also unter R_{min}. Kein Widerstand erfüllt beide Vorgaben. „Ein kleinerer Pull-up löst jeden langsamen Bus“ ist deshalb falsch. Kapazität, Teilnehmeranforderungen oder andere Betriebsbedingungen müssten geändert werden. (1 P)

Die Rechnung behandelt die beiden vorgegebenen Grenzen, keine vollständige Platinenauslegung. Open-Drain darf aktiv LOW ziehen oder freigeben; HIGH entsteht über den Pull-up.

Aufgabe 5: Schreiben und sechs Registerbytes lesen (5 Punkte)

Gegebenes Sensor-Lehrmodell: Sieben-Bit-Adresse `0x68`. Ein Write von `00` nach Register `6B` beendet Sleep. Beschleunigung liegt in sechs während des Lesens stabilen Bytes ab `3B`. SDA bedeutet Serial Data, SCL Serial Clock. ACK ist Acknowledge, NACK Not Acknowledge.

1. Notieren Sie den vollständigen Aufwecktransfer einschließlich Adressbyte, Quittungen und Abschluss. Wer quittiert? (1 P)
2. Notieren Sie den vollständigen Read ab `3B` mit Repeated START. Markieren Sie für alle sechs Datenbytes die Quittung und ihren Erzeuger. (2 P)
3. Berechnen Sie die Zahl der SCL-Takte und deren ideale Mindestdauer für diesen Read bei 400 kHz. Was fehlt in dieser Mindestdauer? (1 P)
4. Unterscheiden Sie unerwarteten Address-NACK, beabsichtigten letzten Read-NACK und Arbitration-Verlust. Warum darf der verlorene Controller nicht blind STOP erzeugen? (1 P)

Musterlösung 5

1. `S D0 A 6B A 00 A P`. S ist START, P STOP, A ACK. `D0 = (68 << 1) | 0` wählt Write. Das Target bestätigt Adresse und beide Schreibbytes. (1 P)
2. `S D0 A 3B A Sr D1 A 10 A 00 A E0 A 00 A 40 A 00 N P` für die Beispielbytes aus Vorlesung 5. Sr ist Repeated START ohne vorheriges STOP. `D1 = (68 << 1) | 1` wählt Read. Das Target bestätigt die drei gesendeten Adress-/Indexbytes. Der Controller bestätigt die ersten fünf gelesenen Datenbytes und gibt beim sechsten für NACK frei. (1 P Phasenfolge, 1 P Quittungsrollen)
3. Neun Bytes mit jeweils neun Takten ergeben 81 SCL-Takte. $81/(400000\,\mathrm{Hz}) = 202{,}5\,\mu\mathrm{s}$. START-/STOP-Bedingungen, Pausen, Stretching und Softwarelücken sind darin nicht als zusätzliche Zeit enthalten. (1 P)
4. Address-NACK ist eine Ablehnung mit mehreren möglichen Ursachen. Letzter Read-NACK ist ein regulärer Abschluss durch den Controller. Bei Arbitration-Verlust gehört der laufende Transfer dem Gewinner; ein STOP des Verlierers könnte ihn unterbrechen. Fehlerpfade benötigen deshalb passende begrenzte Behandlung statt einer pauschalen Aktion. (1 P)

Aufgabe 6: Zahlen, gültige Daten und Leitungspfad (5 Punkte)

1. Dekodieren Sie `10 00 E0 00 40 00` als drei Zweierkomplementwerte mit High-Byte zuerst und skalieren Sie mit 16384 Schritten pro g. Erläutern Sie die Behandlung eines gesetzten Bits 15. (2 P)

2. Betrachten Sie diese reine C-Funktion. `decode_s16_be` ist die in Vorlesung 5 erklärte portable Byte-Dekodierung. Gültige, nicht überlappende Puffer und exklusiver Zugriff sind vorausgesetzt.

   ```
   bool decode_accel_xyz(const uint8_t bytes[6],
                         bool transfer_ok, int32_t out[3]) {
       if (!transfer_ok) return false;
       for (unsigned i = 0; i < 3u; ++i)
           out[i] = decode_s16_be(bytes[2u*i], bytes[2u*i+1u]);
       return true;
   }
   ```

   `out` enthält zunächst `99, -99, 7`. Der Transfer scheitert nach zwei Bytes, `transfer_ok` ist `false`. Welche Ausgabe bleibt erhalten? Sind `00` oder `FF` für sich geeignete Fehlercodes eines Rohdatenbytes? (1 P)

3. Ein differentieller Empfänger sieht zunächst $V_A = 2{,}5$ V, $V_B = 0{,}5$ V. Beide steigen um 0,8 V. Berechnen Sie Differenz und Gleichtaktpegel vorher/nachher. Ist damit jede Störung ausgeschlossen? (1 P)

4. Mehrere langsame Sensoren liegen auf einer Platine und besitzen unterschiedliche Adressen. Begründen Sie eine mögliche Wahl zwischen UART, SPI und I²C und nennen Sie zwei verbleibende Prüfungen. (1 P)

`bool` stammt aus `<stdbool.h>`, die Integer-Typen aus `<stdint.h>`. g ist eine Beschleunigungseinheit, keine Winkeleinheit.

Musterlösung 6

1. `1000` ergibt 4096, `E000` zunächst unsigned 57344, dann $57344 - 65536 = -8192$, `4000` ergibt 16384. Skalierung: $+0{,}25g$, $-0{,}5g$, $+1g$. Bei gesetztem Bit 15 wird vom unsigned zusammengesetzten Wert 65536 subtrahiert. (1 P Rohwerte/Vorzeichen, 1 P Einheiten)
2. Rückgabe `false`, Ausgabe unverändert `99, -99, 7`. Vor dem frühen Return wird kein Byte gelesen oder Ausgabewert geschrieben. `00` und `FF` sind gültige Rohbytes, kein eindeutiger Fehlerersatz. Ein Status muss den gesamten Transfer einschließlich Abschluss bestätigen. (1 P)
3. Vorher $V_d = 2$ V, $V_{\mathrm{CM}} = 1{,}5$ V. Nachher $V_A = 3{,}3$ V, $V_B = 1{,}3$ V, $V_d = 2$ V, $V_{\mathrm{CM}} = 2{,}3$ V. Der erlaubte gemeinsame Pegelbereich bleibt endlich; ungleiche Einkopplung kann die Differenz verändern. (1 P)
4. Beispielsweise I²C: zwei gemeinsam genutzte Signale und Geräteadressierung passen zum Bedarf. Zu prüfen sind etwa Pull-ups/Buskapazität sowie erreichbare Nutzdatenrate/Fristen, elektrische Pegel oder Registervertrag. Unterschiedliche Adressen allein garantieren keine funktionierende Verbindung. Andere begründete Lösungen mit benannten Annahmen sind zulässig. (1 P)

Eine zusammenhängende Lesung garantiert Datensatzkonsistenz nur mit passenden Sensorregeln; deren Stabilität ist hier ausdrücklich vorgegeben.

Optional: Rechenfunktionen am Host prüfen

Übernehmen Sie `decode_s16_be` und `decode_accel_xyz` aus Vorlesung 5 in ein gewöhnliches C-Programm. Prüfen Sie insbesondere:

- Grenzen `0000`, `7FFF`, `8000` und `FFFF`: 0, 32767, −32768, −1.
- Den Sechs-Byte-Datensatz aus Aufgabe 6.
- Fehlgeschlagenen Transfer mit unverändertem Ausgabe-Array.
- Erfolgreichen Transfer auch mit Rohbytes `00` und `FF`.

Verwenden Sie `<stdint.h>` und `<stdbool.h>`, etwa mit `gcc -std=c11 -Wall -Wextra -Werror`. Dies prüft Datenverarbeitung, keine tatsächlichen Quittungsflanken, Sensorbereitschaft oder realen Timinggrenzen.